

SatQuery AI — Project Overview

Single source of truth for the project

Version 1.0 · September 2026 · ISRO SIH 2026 Hackathon Project

How to read this document

This document is the authoritative, codebase-grounded reference for the SatQuery AI project. Everything in it is either:

- **FACT** — confirmed present in the source code (file paths, module names, behavior).
- **PROPOSED** — a suggested next step, explicitly labelled.
- **FUTURE SCOPE** — explicitly deferred, labelled.
- **ASSUMPTION** — an inference, labelled.

Nothing in this document claims capabilities, datasets, API integrations, accuracy numbers, partnerships, deployments, or results that are not verified in the repository.

The critical distinction throughout:

Scope bucket	Meaning
Built (MVP-era, present in code)	Working or wired-up today: FastAPI API, LangGraph orchestrator, 5 specialist agents, evidence agent, report agent, Redis worker, PostGIS models, preview pipeline, GeoJSON coordinate transform, Next.js frontend, Docker Compose stack.
Gated / fallback	Present but dependent on a runtime condition: Gemini VLM online mode (needs <code>GEMINI_API_KEY</code>), user-visible offline/deterministic fallback (always works), planner LLM online mode (needs a key).
Placeholder / stubbed	Declared in the code but not yet implemented: local HuggingFace VLM loader (<code>load_vlm</code>), orchestrator dispatch tools (<code>app/agents/tools.py</code>), per-finding "model_used" provenance.
Proposed	Recommended next step, not present in code.

1. EXECUTIVE PROJECT DEFINITION

1.1 One-sentence definition

SatQuery AI is an agentic, vision–language assistant that lets a user ask a natural-language question about ISRO satellite imagery and get back a grounded answer, georeferenced detections on a GIS map, and a traceable evidence chain — orchestrated by LangGraph across five specialist remote-sensing agents.

1.2 Thirty-second explanation

SatQuery AI is a web application where an analyst — or a non-GIS person — types a plain-English question about satellite imagery, such as "What changed here between 2022 and 2026?" or "Where are the water bodies in this scene?"

A LangGraph **orchestrator** reads the question, picks the correct specialist workflow, and routes it to one of five agents (VQA, captioning, grounding, change detection, optical+SAR fusion). The chosen agent asks a vision-language model (Gemini 2.5 Flash when an API key is present; a deterministic offline heuristic fallback otherwise) to analyze the image, then a geospatial pipeline converts the model's pixel-space detections into real **WGS84 GeoJSON polygons** using the raster's own CRS and affine transform.

An **evidence agent** attaches provenance (source asset, workflow, confidence) to every finding, a **report agent** writes a human-readable summary, and the result is rendered in a Next.js frontend with the detected regions overlaid on the imagery. Everything is asynchronous: queries are queued in Redis, processed by a worker, and streamed to the browser over WebSocket so the user can watch the agents work.

1.3 One-minute explanation

SatQuery AI is a full-stack, production-shaped system rather than a single ML script:

1. **Ingestion** — GeoTIFF/JPEG2000 rasters are uploaded through FastAPI, validated (format, size), georeferenced metadata is extracted (CRS, affine transform, WGS84 bounds) with rasterio, and files are stored either on the local filesystem (`STORAGE_BACKEND=local`) or in S3-compatible object storage (`STORAGE_BACKEND=s3` , MinIO in Docker Compose).
2. **Query submission** — the user creates a session, picks assets, and submits a natural-language query. A Redis-backed async worker picks the job off a queue.
3. **Agentic orchestration** — a LangGraph `StateGraph` runs a planner node (LLM-driven, configurable across openai/gemini/ollama) that selects one of five specialist workflows and crafts a tailored `specialist_prompt`.
4. **Specialist execution** — the specialist agent generates an 8-bit RGB PNG preview of the raster (percentile-stretched for HDR data), sends it to the VLM with a strict JSON-response prompt, receives normalized `[ymin,xmin,ymax,xmax]` boxes in a 0–1000 space, and converts them to WGS84 GeoJSON.
5. **Validation & evidence** — a validation node flags findings below a 0.4 confidence threshold and sets an `uncertainty_flag`; the evidence agent builds a provenance chain and explicitly warns on low-confidence results; the report agent writes a plain-English summary that acknowledges uncertainty.
6. **Frontend** — a Next.js app (React 19, Three.js "Hero" globe, Tailwind) shows the imagery viewer with detection boxes overlaid, a live "Agent Activity" trace panel fed by WebSocket, findings with confidence, and a Reports page with JSON/GeoJSON/PDF export.

The system is deliberately deterministic-by-default: with no API keys configured it still works end-to-end using pixel-differencing heuristics, so demos and tests never fail despite upstream API availability.

1.4 Core problem, core idea, core solution

- **Core problem.** Satellite imagery is hard to interrogate. It requires domain tools (QGIS/ArcGIS), geospatial literacy (CRS, projections, band math, indexing), and — for ML assistance — glue code connecting separate point models (VQA, detection, change detection) that each only handle one task and are rarely georeferenced. The result is a discipline of specialists; a manager, disaster-response officer, or urban planner cannot simply ask a question of an image.
- **Core idea.** Treat satellite-image analysis as **agentic question answering**: one natural-language question, and the system itself plans the analysis, selects the right specialist model/tool, converts every answer into real geographic coordinates, attaches evidence, and explains its reasoning.
- **Core solution.** An agent orchestrator (LangGraph) routing to five specialist vision agents over a common geospatial coordinate-transform pipeline, backed by an async job infrastructure, PostGIS persistence, and a GIS-informed frontend that makes "the answer" and "the regions of the image that justify it" visible side by side.

1.5 Primary USP

Every claim is georeferenced, evidenced, and visibly verifiable. SatQuery does not just answer with text — every finding is a polygon in real WGS84 coordinates linked to its source asset, its confidence, and its location on the imagery, and the user can watch the agent trace that produced it. Uncertainty is never hidden; low-confidence findings are surfaced with explicit warnings.

1.6 What makes it different from a conventional GIS tool

- **Natural language entry point.** No CRS arithmetic, no band math, no layer wrangling to get a first answer; the orchestrator decomposes intent and executes the workflow.
- **Planned, explainable execution.** The system decides *which* analysis (VQA vs grounding vs change detection vs SAR fusion) and shows its reasoning as a trace.
- **Built-in agent + evidence + report layer.** GIS tools render vectors; this system renders *conclusions* with provenance.
- **Test-first determinism.** The same query produces a reproducible pipeline that degrades gracefully when external model APIs are unavailable — a GIS tool has no such notion because it does not depend on a remote model in the first place.

1.7 What makes it different from a generic VLM chatbot

- **Grounding in real coordinates.** Generic vision chatbots return text and at best image-space boxes. SatQuery converts image-space boxes into **pixel** → **native CRS** → **WGS84** GeoJSON polygons using the raster's actual affine transform.
- **Spatially aware data model.** Findings persist with PostGIS geometry (SRID 4326); assets carry CRS/bbox/band metadata. This is a GIS system with an AI front end, not a chatbot with attachments.
- **Five specialist workflows with structured JSON contracts** (see `app/ai/prompts.py`), versus one chat interface.
- **Confidence gating and provenance.** A validation node, explicit `uncertainty_flag`, evidence chains, and warnings rather than confident prose.
- **Deterministic offline fallback.** Works without any model API key.

1.8 Single memorable positioning statement

"Ask any question about any satellite image — and get the answer plotted on a real map, with the evidence to back it up."

2. PROBLEM → IDEA → SOLUTION

2.1 PROBLEM — what is fundamentally difficult today

- **Knowledge is trapped behind specialist tooling.** Meaningful analysis of a satellite image demands a GIS application, coordinate-system competence, and band-level understanding. None of this is accessible in a question.
- **Vision models are task-specific and disconnected.** A VQA model, a grounding model, and a change-detection model each solve one slice, produce outputs in *different* spaces (text, image-space pixels, raw masks), and cannot be composed without engineering glue.
- **Answers are not geographic.** When an AI "looks" at an image for a human, it gives a verbal answer; it rarely tells you *where* on the map the evidence lives, in real longitude/latitude.
- **Trust is missing.** Black-box AI answers about flood zones, deforestation, or urban growth cannot be acted on without knowing the source asset, the confidence, and whether the answer is certain.

2.2 IDEA — the conceptual shift SatQuery introduces

- **From "tools you operate" to "an analyst you ask."** The system owns planning, tool/model selection, execution, and reporting.
- **From "model outputs" to "findings."** Every specialist produces structured, confidence-scored findings that share one schema and one coordinate space.
- **From "image-space" to "ground-truth space."** Detections cascade through pixel → native CRS → WGS84, so a box on the image is a polygon on the map.
- **From "assertion" to "evidence."** Every finding carries `evidence_refs` (source assets), a workflow label, a confidence, and a machine-readable trace; the report agent is prohibited from presenting uncertainty as certainty.
- **From "fragile demos" to "deterministic demos."** Offline fallbacks and cached previews keep the pipeline functional for SIH judging regardless of network/model availability.

2.3 SOLUTION — how the proposed system solves it

- An **asynchronous, full-stack product** (FastAPI + Redis worker + PostGIS + Next.js) that is usable by analysts *and* non-specialists.
- A **LangGraph planner** that classifies intent and routes to exactly one of five specialist workflows with a generated `specialist_prompt`.
- A **geospatial coordinate layer** (`CoordinateTransformer`) that converts VLM boxes (0–1000 normalized, 0–1 relative, or raw pixels) into WGS84 GeoJSON Features with pixel/native/WGS84 bounding boxes under one hood.
- A **validation + evidence pipeline**: 0.4 confidence threshold, uncertainty flag propagation, evidence chains, low-confidence warnings, report-level caution.
- A **GIS-informed UI** where the answer, the image, the detected polygons, and the agent trace are visible simultaneously (and, in Compare mode, pan/zoom-synchronized).

3. USER JOURNEY

A complete walkthrough of a single query, stage by stage.

3.1 Stage table

#	Stage	Input	Processing	Output	Technology / components
1	User / Query	The analyst opens the app, selects one image (Analyze) or two images (Compare), and types a question like "Find all buildings."	The frontend validates that a session and assets exist; the query is submitted.	A <code>query_id</code> returned with HTTP 202; query persisted as <code>status=queued</code> .	Next.js <code>QueryComponent</code> , <code>useAnalysisSessions/{id}/query_sessions.py</code> , <code>QueryWorker</code> .
2	Input & ingestion	A raster is uploaded if not already present.	File validated for format/size, rasters read with rasterio, CRS/bbox/bands extracted; stored on local disk or MinIO/S3.	Registered <code>ImageAsset</code> with metadata; PNG preview cached.	<code>POST /assets/upload/{id}/preview</code> (<code>app/raster_handler.py</code>), <code>PNGHandler</code> , <code>Storage</code> .
3	Understanding	The stored query text + referenced <code>asset_ids</code> + last 5 chat-history turns.	Query text is sanitized against prompt-injection patterns; a Redis queue message is enqueued; the worker loads the query and downloads S3 assets to a temp dir.	The query enters LangGraph with <code>asset_paths</code> pointing at local files.	<code>sanitize_query</code> (<code>security.py</code>), <code>RedisQueue</code> , <code>satquery:query_worker.py</code> , <code>models</code> .
4	Planning	<code>query_text</code> , <code>asset_ids</code> , <code>chat_history</code> .	The <code>plan_node</code> calls the structured-output LLM (<code>PlannerSchema</code>) which emits <code>plan</code> , <code>workflow</code> , <code>requires_validation</code> , <code>specialist_prompt</code> . On any failure, defaults to <code>vqa</code> with <code>requires_validation=True</code> .	A <code>selected_workflow</code> and <code>specialist_prompt</code> ; trace step <code>{"step": "plan", ...}</code> .	<code>app/agents/orchestrator.py</code> , <code>plan_node</code> , <code>PlannerSchema</code> (<code>app/core/model_plugins.py</code>).
5	Model/Tool selection	The planner decision.	<code>route_after_plan</code> dispatches to the matching specialist node; unknown workflows fall back to <code>vqa</code> ; errors go to <code>error</code> .	Execution continues at exactly one specialist node.	LangGraph conditional <code>build_graph()</code> .
6	Analysis	Asset path(s).	Specialist agent: extract raster metadata → make 8-bit RGB PNG preview → call VLM (or offline fallback) with the specialist prompt → parse structured JSON findings.	Structured findings with <code>drop_raw</code> boxes, labels, confidences.	<code>vqa_agent.run</code> , <code>gemma_agent.run</code> , <code>change_agent.run</code> , <code>gemini_client.get_vlm_response</code> (offline fallbacks).

#	Stage	Input	Processing	Output	Technology / components
7	Validation	Findings + planner flags.	If <code>requires_validation</code> , the <code>validation_node</code> finds findings with <code>confidence < 0.4</code> and sets <code>uncertainty_flag</code> . If flagged and there are zero findings, route to <code>error</code> ; otherwise continue. Per the orchestrator, validation is skipped when not flagged (conditional edge goes specialist → evidence).	<code>uncertainty_flag</code> propagated; trace step <code>{"step": "validation", ...}</code> .	<code>validation_node</code> , <code>route_after_spec</code> , <code>route_after_vali</code> , <code>CONFIDENCE_THRES</code>
8	Evidence	Findings.	The <code>evidence_agent</code> builds an evidence record per finding: <code>evidence_id</code> , <code>finding_id</code> , <code>workflow</code> , <code>source_asset_ids</code> , <code>confidence</code> , <code>uncertainty_flagged</code> , <code>query_id</code> , <code>session_id</code> , <code>provenance</code> (tool_calls + placeholder model name); low-confidence entries get an explicit warning.	<code>evidence</code> list.	<code>app/agents/speci</code> , <code>evidence_agent.py</code>
9	Final answer	Findings + evidence + query text + uncertainty flag.	The <code>report_agent</code> invokes the LLM (or a deterministic template if no key) to write a 2–3 paragraph summary that cites findings and acknowledges low confidence. The worker persists the run (<code>AnalysisRun</code> , <code>Finding</code> with PostGIS WKT geometry), the report, and the finished query status; session conversation history is updated.	<code>report</code> dict; <code>run_id</code> ; persisted workflow result, findings, report, evidence.	<code>app/agents/speci</code> , <code>report_agent.py</code> , <code>store_workflow_r</code> , <code>workflows.py</code>), <code>st</code> , <code>routers/reports.p</code>
10	GIS visualization	<code>run_id</code> , session id.	Frontend polls <code>GET .../status</code> and subscribes to WebSocket trace; on completion fetches <code>GET /workflows/{run_id}</code> , projects each <code>bounding_boxes</code> item (pixel or CRS-space) into image space, and draws overlay boxes on the preview.	Detections rendered on imagery; findings and evidence panels populated.	<code>useAnalysisSessi</code> , <code>truth</code> , subscribe for te

3.2 The trust loop the journey encodes

The Analyze page deliberately presents **asset list, image+overlay, and result panel simultaneously** so the user can "check the answer against the pixels while reading it." The Compare page goes further: both frames share a single pan/zoom transform so before/after reads as the same ground, and change detections are drawn on the later frame.

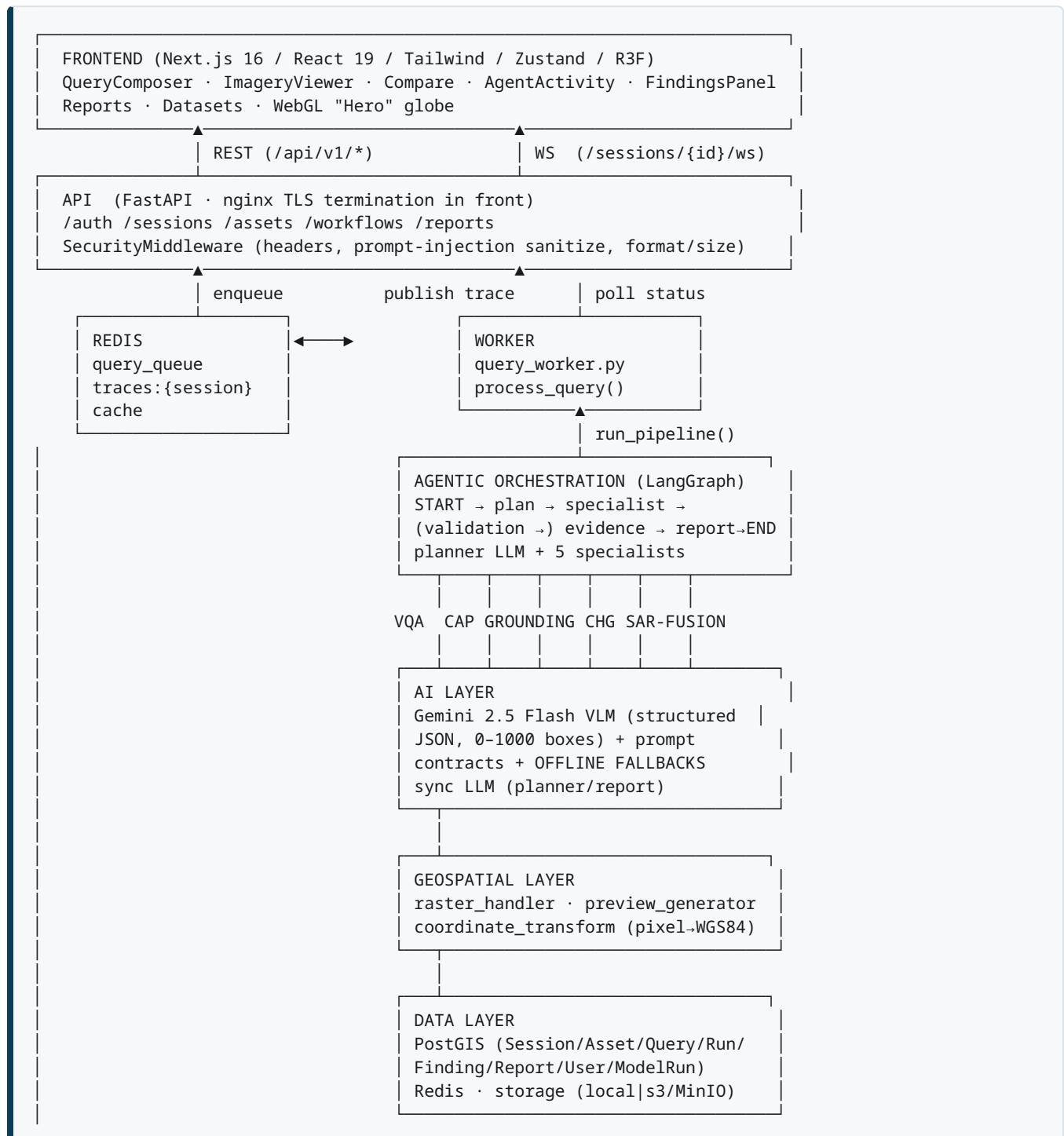
4. COMPLETE SYSTEM ARCHITECTURE

4.1 Human-readable explanation

SatQuery AI is five cooperating layers behind one nginx gateway:

1. **Frontend** — Next.js/React workspace (query composer, imagery viewer with detection overlays, synchronized before/after compare, agent-activity trace stream, reports list and export).
2. **Backend (API + workers)** — FastAPI REST plus a Redis-queued async worker. The API handles sessions/auth/assets/reports; the worker executes the full agent pipeline and persists results.
3. **AI layer** — **one orchestrator LLM** (planner + report writer; GPT-4o by default, swappable to gemini/ollama) and **one VLM** (Gemini 2.5 Flash) shared by all five specialist agents, each driven by a structured JSON prompt contract. There is also a declared-but-stubbed local HuggingFace VLM loader.
4. **Geospatial layer** — raster metadata extraction, preview generation, and the pixel→CRS→WGS84 coordinate transformer that turns model boxes into GeoJSON.
5. **Data layer** — PostgreSQL/PostGIS for metadata and spatial persistence, Redis for queues/cache/pubsub, object storage (local or MinIO/S3) for rasters and derived products.

4.2 Layered architecture



4.3 Component-to-component data flow

1. **frontend** posts a query → `routers/sessions.py:submit_query`
 - sanitizes via `sanitize_query`, persists a `Query(status="queued")`,
 - `redis.lpush("satquery:query_queue", {query_id, session_id})`. Returns `query_id` (202).
2. **worker** (`app/workers/query_worker.py:worker_loop`) `brpop`s the queue, schedules `process_query`.
3. **process_query**:
 - marks query `processing`, publishes `{"step": "worker_started"}` on

`satquery:traces:{session_id}` (WebSocket listeners pick it up).

- resolves assets; **S3 URIs are downloaded to a temp dir** (rasters require local paths); publishes a downstream fallback to the URI if download fails.
- loads session conversation history.
- calls `run_pipeline(...)` → `orchestrator_graph.ainvoke(initial_state)`.

4. LangGraph graph:

- `START` → `plan` → `plan_node` (LLM `PlannerSchema` → `selected_workflow` + `specialist_prompt`).
- `plan` → `specialist` via `route_after_plan`.
- specialist agent returns findings (+GeoJSON) merged into state by `_merge_agent_result`.
- specialist → `validation` (if `requires_validation`) → sets `uncertainty_flag` and routes to `evidence` or `error`.
- specialist → `evidence` (if not flagged).
- `evidence` → `report` → `END`.

5. Back in `process_query` : persist `AnalysisRun`, `Finding` rows (geometry as PostGIS WKT from the first box), `Report`, update query status/trace/conversation history, publish

`{"step": "worker_completed", run_id}`.

6. `frontend useAnalysisSession` : receives completion (WS nudge or status poll), then `GET / workflows/{run_id}` and renders findings over imagery.

4.4 Mermaid diagram

```
flowchart TD
    U[User] -->|open app| F[Next.js Frontend]
    F -->|POST /sessions/{id}/queries| API[FastAPI API]
    API -->|validate + sanitize + persist| PG[(PostgreSQL + PostGIS)]
    API -->|LPUSH satquery:query_queue| RQ[(Redis queues)]
    W[query_worker] -->|BRPOP| RQ
    W -->|s3:// assets -> temp local path| NS[(MinIO / S3)]
    W -->|run_pipeline| GRAPH

    subgraph GRAPH[LangGraph StateGraph]
        PLAN[plan_node] --> ROUTE{route_after_plan}
        ROUTE -->|vqa| VQA[VQA Agent]
        ROUTE -->|captioning| CAP[Captioning Agent]
        ROUTE -->|grounding| GRD[Grounding Agent]
        ROUTE -->|change_detection| CHG[Change Agent]
        ROUTE -->|sar_fusion| SAR[SAR Fusion Agent]
        VQA --> RV{route_after_specialist}
        CAP --> RV
        GRD --> RV
        CHG --> RV
        SAR --> RV
        RV -->|validation if flagged| VAL[validation_node]
        RV -->|else| EV[evidence_node]
        VAL --> REV{route_after_validation}
        REV -->|uncertainty & no findings| ERR[error_node]
        REV -->|else| EV
        EV --> REP[report_node]
        ERR --> END((END))
        REP --> END
    end

    VQA -->|PNG preview + VLM prompt| VLM[(Gemini 2.5 Flash VLM / offline fallback)]
    GRD --> VLM
    CHG --> VLM
    SAR --> VLM

    VLM --> GEO[CoordinateTransformer: pixel-native-WGS84 GeoJSON]
    W -->|persist run/findings/report| PG
    W -->|PUBLISH traces:{session_id}| RC[(Redis pub/sub)]
    API -->|WebSocket /sessions/{id}/ws| FWS[Frontend WebSocket]
    RC -->|read trace| FWS
```

5. AGENTIC ORCHESTRATION

5.1 How the system works (mechanics)

Because this is the heart of the product, everything here is grounded in `app/agents/orchestrator.py` and the specialist modules.

- **How it understands user intent.** The planner (`plan_node`) receives the query text, referenced asset ids, and the last 5 conversation turns. It invokes the LLM with **structured output** (`PlannerSchema`) rather than free text, forcing a `workflow` (one of `vqa` | `captioning` | `grounding` | `change_detection` | `sar_fusion`),

- a `plan` (free-text reasoning), a `requires_validation` boolean, and a generated `specialist_prompt`. If the LLM call throws, the system **defaults to `vqa` with `requires_validation=True`** instead of failing (graceful degradation).
- **How tasks are classified.** The prompt tells the LLM the available workflows plus explicit routing rules ("Prefer geospatial/deterministic operations over model calls"; "Set `requires_validation=true` if the query is ambiguous or confidence may be low"). The offline fallback (used when the planner LLM is unavailable) classifies by keyword: `diff/change/compare/between/before/after` → `change_detection`; `find/locate/where/detect/structures` → `grounding`; else → `vqa`.
 - **How the planner decides capability / selects tools.** Routing is a **single workflow dispatch**, not a dynamic tool loop: `route_after_plan` returns exactly one of the five specialist node names (unknown → `vqa`, error → `error`). The planner also emits the tailored `specialist_prompt`, so the specialist receives both the system prompt (from `app/ai/prompts.py`) and the user-sub-task.
 - **How multiple tools could be chained.** Today, all 8 tools in `app/agents/tools.py` (`reproject_raster`, `crop_raster`, `compute_ndvi` + 5 dispatch stubs) are declared but **not executed** inside the graph; execution lives inside the specialist agents. Chaining them is **proposed** (future), and the planner prompt already encodes the preference to reach for deterministic geospatial tools before model calls.
 - **How outputs are passed.** A single shared `AgentState` TypedDict flows through the graph; `_merge_agent_result` accumulates `intermediate_outputs[agent_name]`, concatenates `findings`, propagates `error`, and appends trace steps.
 - **How validation occurs.** `validation_node` compares each finding's confidence to `CONFIDENCE_THRESHOLD = 0.4`, sets `uncertainty_flag`, and records a warning trace step. The routing rules: specialist → validation only when the planner set `requires_validation`; validation → evidence unless `uncertainty_flag` is set AND the run produced no findings (then error).
 - **What happens when a tool fails.** Specialists catch exceptions and return `{"status": "error", "error": ..., "findings": []}`. The change agent has an extra emergency `_fallback_pixel_diff` (raw pixel-difference thresholding) if the full pipeline throws. A remote (S3) asset that cannot be downloaded falls back to the URI — the agent then fails gracefully with a hint to upload a real file. The report LLM failure falls back to a deterministic summary template.
 - **How loops are prevented.** The graph is a **directed acyclic** StateGraph: there are no cycle edges. Each node appends to the trace, and every conditional edge ends at a terminal (`END` or `error`) within one pass. No node re-enters itself.
 - **How confidence/provenance is maintained.** Confidence is produced by the VLM per finding (or by the deterministic fallbacks) and flows into the `evidence_agent`, which builds one evidence record per finding carrying `source_asset_ids`, `confidence`, `uncertainty_flagged`, `provenance.tool_calls`, and an explicit warning when below 0.4. `evidence_refs` on findings link each finding back to its source asset(s); `ModelRun` records model id/version/input/output/metrics (persisted today, but not yet populated by the pipeline — see 21.5).

5.2 Example workflow 1 — VQA (one image)

- **Query:** "What kind of terrain is this scene?"
- **Intent:** visual question answering / scene interpretation.
- **Planner:** selects `vqa`, generates a `specialist_prompt`, likely `requires_validation=false`.
- **Agent/tool selection:** `vqa_node` → `vqa_agent.run`.
- **Processing:** `extract_raster_metadata` → `generate_rgb_preview` → send preview + `VQA_PROMPT` + question to the VLM (or offline heuristic) → parse `{answer, confidence, supporting_regions[], scene_classification}` → transform supporting region boxes to WGS84 GeoJSON.
- **Evidence:** `evidence_agent` attaches `workflow="vqa"`, `source_asset_ids=[asset0]`, confidence.
- **Answer:** a structured answer + confidence + scene classification + GeoJSON supporting regions rendered with the finding.

5.3 Example workflow 2 — Grounding (one image, open-vocabulary)

- **Query:** "Find all buildings in this scene." (also the first suggestion chip)
- **Intent:** localized entity/feature detection, spatial.
- **Planner:** selects `grounding`.
- **Agent/tool selection:** `grounding_node` → `grounding_agent.run`.
- **Processing:** preview → preview + `GROUNDING_PROMPT` + entity prompt to VLM → parse `{findings:[{label, box_2d, confidence, description}], total_detected, summary}` → each `[ymin,xmin,ymax,xmax]` (0-1000) box → WGS84 GeoJSON Polygon Feature with `pixel_bbox`, `native_bbox`, `is_georeferenced`.
- **Evidence:** per-finding `evidence_refs=[asset0]`.
- **Answer:** list of labeled, confidence-scored, georeferenced regions; frontend draws boxes over the preview and lists them with confidence percentages.

5.4 Example workflow 3 — Change detection (two bi-temporal images)

- **Query:** "What changed between 2022 and 2026 here?" (suggestion chip "What changed here?")
- **Intent:** bi-temporal comparison. The planner must require **two assets**; the Compare page submits `[before, after]`.
- **Agent/tool selection:** `change_detection_node` → `change_agent.run` (requires ≥ 2 assets; the agent errors if fewer are provided).
- **Processing:** metadata from the **before** image (anchors CRS) → two previews → both previews + `CHANGE_DETECTION_PROMPT` to VLM (or offline pixel-diff on downsampled grayscale, quadrant-scored) → parse `{changes_detected, overall_change_level, change_regions:[{change_type, box_2d, confidence, before_state, after_state, description}], summary}` → GeoJSON per region with `before_state` / `after_state` properties.

- **Evidence:** `evidence_refs=asset_ids[:2]` (both the before and after asset).
- **Answer:** summary + change regions; the frontend draws the regions on the **later** frame, synchronized with the earlier frame by a shared viewer transform.

5.5 Example workflow 4 — Multimodal (optical + SAR)

- **Query:** "Identify features using SAR and optical fusion for this region."
 - **Intent:** cross-modal reasoning requiring both sensors.
 - **Agent/tool selection:** `sar_fusion_node` → `sar_agent.run` (optical=asset[0], SAR=asset[1]; errors if < 2 assets).
 - **Processing:** metadata from the optical image (anchors CRS) → preview of both → `[optical_preview, sar_preview]` + `SAR_FUSION_PROMPT` to VLM (or offline SAR backscatter thresholding fused with optical brightness) → parse `{fused_findings: [{label, box_2d, confidence, optical_evidence, sar_evidence, fusion_interpretation}], fusion_summary, overall_confidence}` → GeoJSON per finding.
 - **Evidence:** `evidence_refs=asset_ids[:2]` — the multimodal claim is tied to both sensors.
 - **Answer:** fusion summary + fused findings, each describing what *each* modality contributed and the combined interpretation (e.g., "Urban Double-Bounce Zone," "Flooded / Inundated Area," "Open Water Body," "Bare Soil / Cleared Land").
-

6. CORE AI CAPABILITIES

Placeholder / gated status is noted per capability. **Nothing here claims an accuracy number.**

6.1 Visual Question Answering (VQA)

- **Purpose:** answer a user's question grounded in the visual content of one image.
- **Input:** one local raster path; user question.
- **Model/tool category:** VLM (online: Gemini 2.5 Flash; offline: deterministic heuristic which returns a canned answer + supporting region + scene classification).
- **Output:** `{answer, confidence, supporting_regions[{label, box_2d, description}], scene_classification}` + WGS84 GeoJSON for each supporting region.
- **Where it fits:** `vqa_agent.run` (also reused by `run_captioning` with the question "Describe this satellite image in detail.").
- **MVP feasibility:** HIGH — fully implemented; the only requirement is an asset and, optionally, an API key.
- **Evaluation method (proposed):** hand-labeled scenario panels; human rating of answer correctness and supporting-region placement; measure n/10 for a standard panel of ISRO-style scenes.

6.2 Scene understanding / captioning

- **Purpose:** produce a natural-language description of a scene.
- **Input:** one raster.
- **Model/tool category:** VLM (same client; `captioning_node` rewrites the query and relabels findings as `captioning`).
- **Output:** a descriptive answer + scene classification + supporting regions.
- **Where it fits:** `orchestrator.captioning_node`, `vqa_agent.run_captioning`.
- **MVP feasibility:** HIGH — shares the VQA implementation (no dedicated captioner model; documented).
- **Evaluation method (proposed):** qualitative coherence rating; caption vs. reference comparison on a small labeled set.

6.3 Text-guided grounding (open-vocabulary localization)

- **Purpose:** locate arbitrary requested entities ("buildings," "water bodies," "roads," "ships") on an image.
- **Input:** one raster + entity/text prompt.
- **Model/tool category:** VLM (open-vocabulary) returning 0–1000 normalized boxes; offline fallback returns fixed labeled boxes.
- **Output:** findings with `label`, `confidence`, WGS84 GeoJSON polygons.
- **Where it fits:** `grounding_agent.run`; frontend renders boxes on `ImageryViewer`.
- **MVP feasibility:** HIGH — implemented end to end.
- **Evaluation method (proposed):** IoU vs. token-level segmentation masks/boxes for chosen classes on a small labeled set (e.g., building = high-reflectance geometric clusters) + human inspection.

6.4 Bi-temporal change detection

- **Purpose:** identify what changed between two acquisitions of the same area.
- **Input:** two registered rasters (same CRS/extent; pre-registered per the agent's documented contract).
- **Model/tool category:** VLM pairwise comparison; online mode yields change classes
- before/after descriptions; offline mode thresholds absolute pixel differences (mean diff > 15 ⇒ change; top-2 quadrant sectors above 10). Emergency fallback: `mean + 2σ` threshold on band 1.
- **Output:** `changes_detected`, `overall_change_level`, `change_regions[]`, summary; GeoJSON per region with `before_state` / `after_state`.
- **Where it fits:** `change_agent.run`; Compare page.
- **MVP feasibility:** HIGH for demo data (the sample 2022/2026 pair has a synthetic urban-expansion footprint); geospatial alignment of arbitrary real pairs is a known limitation (see 12.4).
- **Evaluation method (proposed):** change-region hit rate vs. known/synthetic change maps; class confusion; qualitative before/after plausibility.

6.5 Optical + SAR reasoning

- **Purpose:** fuse SAR backscatter with optical context to disambiguate features neither modality alone confirms.
- **Input:** optical + SAR rasters over the same area.
- **Model/tool category:** VLM cross-modal prompt; offline fallback does backscatter-intensity thresholding (top-35% = strong scatterers) fused with optical brightness per quadrant into classes (double-bounce, flooded, open water, bare soil).
- **Output:** `fused_findings[]` each with `optical_evidence` , `sar_evidence` , `fusion_interpretation` , `overall_confidence` , GeoJSON.
- **Where it fits:** `sar_agent.run` .
- **MVP feasibility:** MEDIUM–HIGH. Flow exists, but SAR interpretation quality with the generic VLM approach is unproven; sample SAR is synthetic (VV/VH random+double-bounce block). Flagged as a mitigation risk (12.3).
- **Evaluation method (proposed):** cross-modal agreement metric (do the two evidence strings agree?), class accuracy on the synthetic sample, human rating of `fusion_interpretation` .

6.6 Evidence & validation

- **Purpose:** convert "model said X" into "finding X with source, confidence, workflow, and uncertainty flag"; keep claims checkable.
- **Input:** findings list, uncertainty flag, query/session ids, trace.
- **Model/tool category:** deterministic (no model).
- **Output:** `evidence[]` records with `provenance.tool_calls` + explicit low-confidence warnings.
- **Where it fits:** `evidence_agent.run` , `validation_node` .
- **MVP feasibility:** HIGH — implemented deterministically.
- **Evaluation method (proposed):** checklist audit — every finding must have `evidence_refs` , `confidence` , and `uncertainty_flagged` ; every low-confidence result must carry a warning; report must acknowledge the flag (testable assertion, see 19.7).

7. DATA & DATA PIPELINE

7.1 Actual data present (verified)

Data	Location	Notes (verified in code)
ISRO optical 2022 (sample)	<code>sample_data/</code> <code>isro_optical_2022.tif</code> , also <code>satquery_backend/sample_data/</code>	Synthetic 512×512, 3-band uint16, EPSG:32643, near Hyderabad/NRSC footprint (per <code>scripts/generate_sample_geotiffs.py</code>); 2022 = natural terrain/agricultural

Data	Location	Notes (verified in code)
ISRO optical 2026 (sample)	<code>sample_data/isro_optical_2026.tif</code>	Same frame, 2022 + synthetic built-up block (rows 40–180, cols 280–460) + road corridor (cols 360–375)
ISRO SAR 2026 (sample)	<code>sample_data/isro_sar_2026.tif</code>	Synthetic dual-band VV/VH uint16 with elevated double-bounce values in the built-up block
Raw uploads	<code>data/raw/*.tif</code>	Multiple uploaded rasters (date-stamped 2026-09-08 in this checkout)
Previews	<code>data/previews/*.png</code>	Generated 8-bit PNG previews (optical + SAR variants)
Derived products	<code>data/derived/*.json/.geojson/.pdf</code>	Report exports, cached previews per asset (<code><id>_1024.png</code>)
Seeded demo rows	PostgreSQL via <code>scripts/seed_demo.py</code>	2 assets (optical 2026 + SAR 2026), session, 2 queries (change detection, SAR fusion), 2 runs, 4 findings with PostGIS geometry, 2 reports — with deterministic <code>demo...</code> ids

Important honesty note: every raster in this repository is **synthetic**, generated by `scripts/generate_sample_geotiffs.py` (random bands + injected blocks). The SAR is not real Sentinel-1 GRD data. The pipeline is real; the contents of these images are illustrative demo terrain. Real datasets are deliberately **injected manually** (see the `DATASET INJECTION POINT` blocks throughout the code) — SatQuery contains no downloader/scrapper by design.

7.2 Pipeline

```
Satellite imagery (GeoTIFF)
| user upload (validation: format, ≤500MB) or manual drop in data/raw/
▼
Preprocessing
| rasterio: CRS, affine transform, bounds → WGS84 extent, bands, dtype
| PreviewGenerator: percentile (2-98%) stretch → 8-bit RGB PNG (≤1024px)
▼
Normalization / tiling
| read_raster_bands: downsampled band arrays (bilinear) for analysis;
| no tiling today (proposed for large scenes)
▼
Model input
| PNG preview(s) → VLM (Gemini 2.5 Flash) with specialist JSON prompt,
| or → offline heuristic fallback (deterministic)
▼
Inference
| structured JSON: answers, 0-1000 normalized boxes, confidences
▼
Spatial reconstruction
| CoordinateTransformer: normalize_box → pixel → native CRS (affine) →
| WGS84 (pyproj) → GeoJSON Polygon Feature with pixel/native/WGS84 bboxes
▼
Visualization
| GeoJSON + preview rendered in ImageryViewer; findings bordered; report
| export (JSON/GeoJSON/PDF); PostGIS geometry persisted for queryability
```

7.3 Data categories

- **ESSENTIAL (must exist before any query):** at least one georeferenced raster as a local path; the asset registered in Postgres with CRS metadata; the PNG preview. For change detection / SAR fusion, two co-registered rasters.
- **OPTIONAL:** API keys (Gemini/OpenAI) — the system runs without them via fallbacks.
- **CACHEABLE (already cached):** PNG previews (keyed `{asset_id}_{max}.png` in `data/derived/`), previews re-generated only when the source is newer; no Redis-side caching of analysis results yet (multiple `set_cache` / `get_cache` helpers exist but are not yet used by the pipeline — **proposed**).
- **API access needed:** none for built-in demo flow. For the *online* model modes you need a Google Gemini API key (`GEMINI_API_KEY`) and/or OpenAI key for the planner (`OPENAI_API_KEY` / `GOOGLE_API_KEY` / local Ollama), configured via `.env` or Docker secrets (`secrets/*.txt`). No satellite-data API is required by design.

8. TECHNOLOGY STACK

All items below appear in the actual codebase (`requirements.txt`, `frontend/package.json`, `docker-compose.yml`, source files).

Layer	Technology	Purpose	Why chosen	MVP/ Future
Frontend	Next.js 16.3 (React 19), TypeScript	App shell, routing, SSR/prerender	FastAPI-backend pairing; <code>app/</code> directory + React 19 ecosystem; typed via <code>types.ts</code> mirroring backend schemas	MVP (current)
Frontend	Three.js + React Three Fiber + drei + postprocessing	WebGL "Hero" globe landing scene	Visual anchor for the demo; real ISRO/ Cartosat-3 narrative (in comments); proves the product is production-polished, not a script	MVP (current)
Frontend	TailwindCSS 4, Framer Motion	Theming (light/dark), animations, layout	Fast iteration; design tokens in <code>lib/theme.ts</code> ; theme store in Zustand	MVP (current)
Frontend	Zustand	Client state (appearance, scene, earth textures)	Minimal, framework-light global state	MVP (current)
Backend	Python 3.11 + FastAPI 0.115	REST API + WebSocket	Async, typed, auto-docs (<code>/docs</code>), ASGI — matches an async agent/ worker stack	MVP (current)
Backend	LangGraph 0.2.35 + LangChain 0.3	Agentic orchestrator (planner graph, state, conditional routing)	Explicit graph + typed state + async execution; avoids hand-rolled agent plumbing	MVP (current)
Backend	Uvicorn / gunicorn (uvicorn workers)	ASGI server	Standard FastAPI production deployment	MVP (current)
AI/ML	langchain-openai <code>gpt-4o</code> (default orchestrator LLM)	Planner + report synthesis	Best production results for agentic tasks (comment in code); hot-swappable to gemini/ollama	MVP (gated — needs key; stub LLM otherwise)
AI/ML	<code>google-genai</code> Gemini 2.5 Flash	Specialist VLM for all five workflows	Native multimodal + structured JSON output (temperature 0.2, <code>response_mime_type=application/json</code>)	MVP (gated — needs key; offline heuristics otherwise)
AI/ML	HuggingFace <code>transformers</code> (AutoModelForVision2Seq)	Local VLM fallback (BLIP-2, LLaVA...)	Declared in <code>model_provider.load_vlm</code>	Stub/not wired today
AI/ML	<code>opencv-python-headless</code> , Pillow, numpy	Preview normalization, offline pixel differencing	Deterministic, no-API fallbacks	MVP (current for CV heuristics)
Geospatial	rasterio 1.3.11, GDAL (system lib)	Raster IO, CRS, affine, bounds, WGS84 transform	Industry-standard raster stack	MVP (current)

Layer	Technology	Purpose	Why chosen	MVP/ Future
Geospatial	pyproj, affine, shapely, geopandas, fiona	Projection transforms, geometry, GeoJSON	Native CRS↔WGS84 conversion + GeoJSON construction	MVP (current)
Data	PostgreSQL 16 + PostGIS 3.4 (asyncpg, SQLAlchemy async, geoalchemy2)	Metadata + spatial persistence (<code>Geometry(4326)</code> , <code>ST_AsGeoJSON</code> , <code>ST_XMin</code> ...)	Geospatial queries natively; single async ORM	MVP (current)
Data	Alembic	Schema migrations	<code>docker-compose</code> runs <code>alembic upgrade head</code>	MVP (current)
Data	Redis 7.4 (redis-py asyncio, tenacity retries)	Query queue, trace pub/sub, cache helpers	Async job queue + lightweight pub/sub for trace streaming	MVP (current; cache helpers unused yet)
Storage	Local filesystem backend; boto3 + MinIO/S3-compatible backend	Persist uploads + derived products; presigned URLs	<code>STORAGE_BACKEND=local</code> for dev/ MVP, <code>s3</code> for distributed production	MVP (current, local); S3 = future
Deployment	Docker Compose (8 services: nginx, migrate, api, worker, frontend, db, redis, minio, + minio-init)	Reproducible full stack	One-command bring-up; migrations/ secrets/service-health orchestrated	MVP (current)
Deployment	nginx (alpine) reverse proxy, TLS, WS upgrade	HTTPS gateway, WebSocket upgrade, 500MB uploads	Single public endpoint; TLS termination for SIH demo	MVP (current)
Ops	Django-style secrets via Docker Secrets (<code>secrets/*.txt</code> → <code>/run/secrets</code>)	Key/secret injection, prod guardrails (<code>ALLOWED_ORIGINS</code> , <code>SECRET_KEY</code> checks)	Credentials not in <code>.env</code> for production	MVP (current)
Observability	structlog, <code>SecurityMiddleware</code> (request id, process time, security headers, rate limiting), <code>metrics.py</code> (p50/p95 latency hooks)	Structured logs, audit events, perf tracking	Debuggability + NFR "performance tracking"	MVP (current; metric aggregation proposed)

9. MVP IMPLEMENTATION PLAN

9.1 Definition: the smallest technically convincing MVP

One uploaded (or seeded) pair of rasters + one natural-language question + a routed specialist workflow + a WGS84-georeferenced finding + evidence + a report + a frontend overlay — all in Docker Compose, working without any external API key.

The witness line: from `docker compose up` to "Find the water bodies in this image" rendering two polygons over the preview with confidence and an evidence chain — reproducibly, with live agent traces visible.

9.2 MUST HAVE (current repository already delivers nearly all of this)

- FastAPI API: sessions, query submission (async, HTTP 202), status polling, WebSocket traces, assets (upload/list/metadata/preview), workflows, reports, export.
- LangGraph planner with structured output + 5 specialists + validation/evidence/report nodes + graceful error node.
- Georeferencing: `CoordinateTransformer` producing WGS84 GeoJSON with pixel/native/WGS84 boxes; `raster_handler` metadata; preview pipeline.
- Deterministic offline fallbacks for every specialist (zero-API-key demo).
- Persistent PostGIS models + Alembic migrations + seed script.
- Redis queue + worker + pub/sub trace streaming.
- Next.js workspace: Analyze, Compare (synchronized viewers), Datasets, Reports, hero.
- Docker Compose stack (nginx wrapper, TLS).

9.3 SHOULD HAVE (proposed for SIH-quality completeness)

- **Report agent on the demo panel:** give the planner/report writer a disposable `OPENAI_API_KEY` for *one* run vs. the offline fallback (A/B visual).
- **Region "Verify" affordance:** click a finding → zoom the overlay to its bbox (today boxes are toggle-highlightable; zoom-to-finding is proposed).
- **Ground-truth accuracy statement** in the demo README (which panels are FIXED expected outputs).
- Pipeline-level **latency + memory logging** aggregation (the `metrics.py` hooks exist but data is only in logs).
- **Redis caching** of repeated identical queries (helpers exist, unused).

9.4 FUTURE (explicitly out of MVP)

- Real HuggingFace local VLM (`load_vlm`).
- Real tool chaining (`reproject_raster` , `crop_raster` , `compute_ndvi`) driven by the planner.

- Tiling for large scenes; multi-temporal trend analysis; batch ingestion.
- Authenticated multi-user workspaces (auth exists for login, sessions are per-tab).
- Model-weight downloader / remote catalogue search (explicitly forbidden in `PROMPT.md` MVP NOTE).

9.5 Definition of done for the MVP

A single `docker compose up` produces: ① a reachable frontend + API; ② a seeded catalogue with the 2022/2026 optical + SAR samples; ③ a successful VQA query, a grounding query, a change-detection query (Compare), and a SAR-fusion query; ④ every result has `confidence`, `evidence_refs`, GeosJSON, and a report; ⑤ the frontend shows boxes over imagery plus the agent trace; ⑥ no API key is required for ③–⑤.

10. DEMO SCENARIO (SIH — 3–5 minutes)

10.1 Recommended scenario: "Ask, watch, verify"

Opening (0:00–0:30)

"This is SatQuery AI — ask any question about any satellite image. Watch what happens behind an answer." Pull up the landing hero (the 3D globe with a Cartosat-3-style satellite), then enter the Analyze workspace with the seeded optical-2026 asset.

Input (0:30–1:10)

Type: **"Find all buildings in this scene."** (grounding — the first suggestion chip). Submit. Point at the Agent Activity panel: plan → worker started → grounded → evidence. Make the point: "the system *decided* this was a grounding task, generated its own specialist prompt, and ran it."

Processing (1:10–1:50)

The trace streams live over WebSocket (or is reconstructed by polling when offline — the UI labels "Streaming" vs "Polling — trace socket unavailable" honestly). Click between detections; watch `bounding_boxes` overlay appear on the preview with confidence badges.

Output (1:50–2:30)

The report panel summarizes; per-finding confidence is displayed; the evidence chain lists source asset + workflow. **Pivot to the *where* claim:** "the model answered in

pixels — here the geospatial pipeline turned those pixels into real coordinates." Show the GeoJSON export (three formats: JSON, GeoJSON, PDF) and note WGS84 footprints.

Evidence (2:30–3:30)

Switch to **Compare** with 2022 → 2026 and run "What changed between these?" Draw the change regions on the later frame; both viewers are pan/zoom-synchronized so the eye aligns the same ground. Note the `before_state` / `after_state` properties per region.

Wow moment (3:30–4:30)

Kill the internet (or remove the API key) and ask a question — it still answers, on the map. Run an Optical+SAR fusion query in the same session and show that the answer cites *both* sensors ("strong SAR backscatter co-located with high optical reflectance confirms dense urban area") with `evidence_refs = [optical, sar]`. Close on the positioning line: "the answer is a mapping, not a chat bubble — and it never breaks on stage."

Optional final flourish

Stay on a low-confidence example (a query that triggers `requires_validation`) and show the **warning** explicitly surfaced in the report: "One or more results have low confidence. Do NOT present as certain."

10.2 Alternative scenarios

- **Scenario B — Disaster-style urgency (recommended variant):** frame the same three runs around a flood response: "Map the water bodies" (grounding), "What changed after the event?" (change detection), "Confirm flooded area where optical can't" (SAR fusion). Strong narrative for judges; reused components.
- **Scenario C — Data-quality/"honest AI" pitch:** lead with the confidence gate and the offline-fallback determinism, then demonstrate all five workflows. Best for technical judges; riskier for live wow because it front-loads caveats.

10.3 Recommendation

Scenario A (10.1), with the flood framing as the fallback wording. It hits every judging axis in five minutes: interaction (NL), intelligence (agent routing across five workflows), geospatial correctness (GeoJSON), explainability (trace+evidence), and robustness (offline fallback). The internet-cut "wow" requires a **dry-run rehearsal** — de-integrate keys before the demo, do not rely on removing them mid-talk.

11. FEASIBILITY ANALYSIS

11.1 Technical feasibility — HIGH

The end-to-end pipeline is **already implemented and wired**: API, queue, worker, graph, specialists, geo pipeline, persistence, frontend, Docker. The main remaining engineering is marginal: hardening, caching, real-data validation, and demo polish. Bottlenecks: no real (non-synthetic) rasters yet; S3 mode not exercised on a real external bucket for the API service (MinIO used locally); local VLM loader unused.

11.2 AI/ML feasibility — MEDIUM for quantifiable accuracy, HIGH for flow

- The *flow* (prompt → JSON → coordinates) is proven and robust to model choice.
- The *quality* of vision answers on real ISRO data is **unproven** (only synthetic panels so far). Gemini-class VLMs handle scene ops well; open-vocabulary grounding at fine granularity and SAR physics are the realistic weak spots.
- **No accuracy numbers exist anywhere in the code.** Every evaluation approach in section 19 is proposed.

11.3 Data feasibility — MEDIUM-HIGH

- Pipeline is data-agnostic (any CRS/band count handled by the preview + transform layers). But: **co-registered before/after pairs are required** for change detection, and **no real public satellite pairs are packaged** (by design — manual injection). Obtaining registered pairs of the *same real area* is the top external dependency.

11.4 Infrastructure feasibility — HIGH

- Single-node Docker Compose is sufficient for the SIH demo; the API/worker are each capped at 2 CPU / 2GB in Compose; PostgreSQL + PostGIS + Redis + MinIO all local.
- S3-backed horizontal scaling is designed for (storage abstraction), not yet proven at scale.

11.5 Integration feasibility — HIGH for what exists; MEDIUM for online modes

- Backend/frontend integration is tight and typed (schemas mirrored in `frontend/src/lib/api/types.ts`).
- Online model integration needs stable external keys during the demo; the offline path removes that risk.
- The one structural debt: **routers require local raster paths**; S3 requires a per-run download handshake (already implemented in the worker).

11.6 Overall rating: ****HIGH for building and demoing; MEDIUM for production-ready**

vision quality** — with the explicit caveat that no quantitative evaluation exists yet.
The honest pitch to judges: "a complete, robust architecture + a reproducible pipeline;
the next sprint is evaluation-grade data and model tuning."

12. CHALLENGES & MITIGATION

#	Challenge	Impact	Probability	Mitigation	MVP strategy
12.1	Model hallucination (VQA/grounding answers detached from pixels)	High — undermines trust, the core promise	Medium	Confidence gates (0.4), validation node, uncertainty flag, evidence chain, report-level warnings; offline heuristic = fully deterministic; planner prompt prefers deterministic ops	Ship with conservative <code>requires_validation</code> default; make low-confidence warnings a visible demo point
12.2	GPU / inference limitations (local VLM or heavy models on CPU)	Medium — latency, and local-model feasibility	Medium	API-based VLM (Gemini) avoids local GPU entirely; local <code>load_vlm</code> is stubbed and not on the demo path; previews capped at 1024px	Keep <code>VLM_DEVICE=cpu</code> off the demo path; use the API model or offline fallback
12.3	SAR complexity (physics: speckle, incidence angle, calibration)	High for correctness	High on real data	Offline SAR fusion uses coarse quadrants + thresholds, not real calibration; VLM prompt treats SAR qualitatively	Demo with synthetic VV/VH sample ; label SAR as "illustrative"; do not claim calibrated backscatter metrics
12.4	Geospatial alignment (two rasters must share CRS/extent for change detection)	High	Medium	Agent contract: "same CRS and spatial extent (or pre-registered)"; error paths when not met; Compare frontend visually aligns via shared transform	Use same-extent synthetic pair for demo; document pre-registration requirement
12.5	API/data instability (Gemini/OpenAI outages, rate limits, key revocation)	High — demo-killing	Medium	Offline fallback for every specialist; worker-side try/except→graceful; stub LLM for planner/report; Redis retries via tenacity	Run the demo with keys already removed; verify the full pipeline offline

#	Challenge	Impact	Probability	Mitigation	MVP strategy
12.6	Agent failures (planner misroutes, structured-output parse failure)	Medium	Medium	<code>plan_node</code> exception → default vqa + validation; <code>route_after_plan</code> fallback to vqa; error node; trace records each step	Exercise the default-vqa path in rehearsal
12.7	Latency (LLM + VLM round trips for one query)	Medium — compound demo lag	Medium	Everything async (202 + queue); streaming traces mask wait; cached previews; 2-min frontend timeout w/ clear messaging	Demo on warm data; pre-generate previews; keep 1–2 queries live, not 5
12.8	Scope creep (post-MVP features: tiling, multi-temporal, auth tiers, batch ingest)	High — dilutes the demo	High	Locked MVP scope in <code>PROMPT.md</code> ("DO NOT BUILD"); this doc re-lists out-of-scope explicitly; frontend already pages reflect "Reports scoped per tab"	No new features until SIH; freeze branch after demo
12.9	Data legitimacy (only synthetic rasters)	Medium — credibility	High	Present the generator openly (<code>scripts/generate_sample_geotiffs.py</code>); position real-data injection as the documented path	If real ISRO/Bhuvan data is available, inject 1–2 real GeoTIFFs for the demo

13. VIABILITY

13.1 User need

Real and visible: disaster-response analysts, urban planners, agriculture observers and researchers all hit the same wall — "I have an image, I have a question, I do not have a GIS degree or a modelling team." SatQuery lowers the bar from "operate QGIS + chain models" to "ask."

13.2 Practical utility

Direct utility measured in lines of friction removed: a question replaces CRS configuration, band math, model selection, and post-hoc coordinate conversion. The compare workflow reduces before/after alignment effort, and the export layer (JSON/GeoJSON/PDF) means findings are consumable downstream in any GIS.

13.3 Scalability

- Pipeline: shared-nothing worker model (Redis queue + worker) scales horizontally behind the storage abstraction.
- Data: object storage for rasters; PostGIS for spatial queryability; preview caching already built.
- Computing vision: API-VLM outsources GPU; a local VLM would require GPU — deferred.

13.4 Domain applicability

The five workflows map directly onto ISRO-focused domains: disaster management (change detection, SAR flood confirmation), agriculture (NDVI tool declared; VQA over multispectral), urban planning (grounding, change detection), infrastructure (SAR double-bounce vertices), environment (deforestation/vegetation change), and research/governance (reports + GeoJSON handoff).

13.5 Competitive differentiation

See section 15. The durable differentiators are: multi-model agentic routing from NL, georeferenced outputs everywhere (not image-space), explicit evidence/uncertainty discipline, and the offline-deterministic resilience — a demo can never fail on stage.

13.6 Long-term potential

With real data + evaluation on top, the architecture is a credible foundation for a general **remote-sensing question-answering platform** (add more specialists, more tool chains, batch pipelines, user accounts). The current weakness — no eval — is the explicit next investment (section 19).

14. IMPACT & BENEFITS

14.1 BENEFITS (direct user value)

- **Accessibility:** non-GIS users get defensible answers without learning coordinate systems or band math.
- **Speed:** one question instead of a GIS workflow; async processing and streaming traces keep interaction feel live.
- **Explainability:** every finding has a source asset, workflow, confidence, and a machine-readable trace — users can audit rather than trust.
- **Portability:** JSON/GeoJSON/PDF export drops straight into existing GIS and reporting pipelines (Findings page has a live export path).

- **Reliability:** zero-key offline fallback means the tool works in constrained environments (schools, government offices, demos).

14.2 IMPACT (larger-scale outcomes)

- **Disaster management:** natural-language flood/cyclone questions answered in minutes; SAR+optical fusion (per `sar_agent`) targets exactly the "flooded but cloudy" case where optical alone fails.
- **Agriculture:** NDVI computation exists as a declared geospatial tool; change detection over fields supports crop-cycle and land-conversion monitoring.
- **Urban planning:** grounding finds built-up regions; change detection measures expansion between epochs (the synthetic 2022→2026 pair literally demonstrates urban growth).
- **Infrastructure:** SAR double-bounce signatures flag built surfaces and loading structures that optical geometry alone cannot confirm.
- **Environment:** deforestation/vegetation-loss change classes are in the prompt contract (`change_type: Deforestation, Vegetation Loss`).
- **Research & governance:** evidence chains + GeoJSON exports turn AI output into an auditable record usable in academic or administrative review.

*All of the above should be presented as **capabilities the architecture enables**, not as shipped deployments.*

15. COMPETITIVE DIFFERENTIATION

Capability	Traditional GIS (QGIS/ArcGIS)	Single vision AI model	Generic VLM chatbot (ChatGPT Vision)	
Natural-language interaction	None (tool-driven)	None (script/CLI)	Strong	Full — planner decomposes intent
Agentic routing across specialists		One task	Chat only	LangGraph planner → exactly one of 5 workflows
Multiple specialist models	Toolbox ≠ models	One		VQA/captioning/grounding/change/SAR agents (one VLM + distinct prompts today)
Temporal / bi-temporal analysis	Manual		Weak	Dedicated change-detection workflow + synchronized Compare UI
Optical + SAR cross-modal reasoning	Manual		Weak	SAR-fusion workflow with per-sensor evidence

Capability	Traditional GIS (QGIS/ArcGIS)	Single vision AI model	Generic VLM chatbot (ChatGPT Vision)	
Evidence grounding / provenance	Attribute data		Opaque	Evidence agent, <code>evidence_refs</code>, workflow labels, trace
Georeferenced output (WGS84 GeoJSON)		Image-space	Image-space	Pixel→CRS→WGS84 via <code>CoordinateTransformer</code>
GIS visualization of detections				Overlay boxes on imagery; synchronized before/after viewer
Confidence gating + uncertainty surfacing		Some		0.4 threshold, <code>uncertainty_flag</code>, explicit warnings, never hidden
Runs with zero API keys	n/a	Depends		Deterministic offline fallbacks end to end

16. INNOVATION

16.1 Agentic orchestrator over satellite workflows

- **Existing limitation:** users choose the model/tool; mischoice wastes effort.
- **SatQuery approach:** LangGraph planner emits `workflow` + `specialist_prompt` with structured output, and routes through a defined graph (plan→specialist→validation→evidence→report).
- **Why it matters:** turns "which tool do I need?" into "ask the question," and makes the reasoning observable.

16.2 VLM pixels → WGS84 ground truth

- **Existing limitation:** vision models return image-space boxes; upstream GIS integration is manual glue.
- **SatQuery approach:** `CoordinateTransformer.normalize_box` (0–1000 / 0–1 / raw pixels) → affine to native CRS → pyproj to WGS84 → GeoJSON Feature with pixel/native/WGS84 bboxes.
- **Why it matters:** the answer *is a map feature*, clickable, exportable, and queryable in PostGIS — the core bridging idea of the project.

16.3 Evidence-chain discipline with never-hidden uncertainty

- **Existing limitation:** LLM answers present uncertainty as assertion.
- **SatQuery approach:** validation node at 0.4, `uncertainty_flag` propagated through evidence to report; evidence agent emits explicit "Do not treat as ground truth"

warnings; report prompt is constrained to "Do not make up facts not present in the findings."

- **Why it matters:** this is the trust property that lets real agencies act on output.

16.4 Deterministic offline remote-sensing pipeline

- **Existing limitation:** demos fail when API keys/vouchers run out.
- **SatQuery approach:** pixel-differencing + backscatter-threshold heuristics for change detection / SAR fusion / grounding/VQA, and a stub-safe LLM path — all without a key (documented in `gemini_client.py`).
- **Why it matters:** resilience on stage and in constrained networks; also gives the pipeline a deterministic baseline for evaluation.

16.5 Synchronized bi-temporal GIS viewer

- **Existing limitation:** comparing before/after means manual alignment, turning change detection into an error-prone eye test.
- **SatQuery approach:** Compare page shares one pan/zoom transform (`ViewerTransform`) between both frames; detections are drawn against the later frame so "change footprint = where something now is (or now isn't)."
- **Why it matters:** the *observation* and the *conclusion* stay visually co-registered — this is the UI equivalent of the geographic correctness the pipeline provides.

17. IMPLEMENTATION ROADMAP

Phase	Theme	Deliverables	Dependencies	Expected output	Risks
0 (done)	Foundation	FastAPI + PostGIS + Redis + storage abstraction + auth + Docker Compose + nginx + Alembic	—	Full-stack baseline	—
0 (done)	Core AI workflows	Specialist agents + VLM client + offline fallbacks + coord transform + previews	Foundation	VQA/grounding/change/SAR end-to-end	—
1 (current)	Agentic orchestration	LangGraph planner routing to specialists, validation, evidence, report	Core workflows	explainable multi-workflow routing	planner misroutes; structured-output parse failures (mitigated by defaults)
2 (current)	Evidence & GIS	Evidence chain, PostGIS findings, GeoJSON exports, frontend overlay, Compare viewer	Phase 1	Checkable, exportable results	PostGIS geometry persistence quirks (see 21.7)

Phase	Theme	Deliverables	Dependencies	Expected output	Risks
3	Real data & evaluation	Inject real ISRO/Bhuvan rasters; build a labeled evaluation panel; measure VQA/grounding/change	Phase 2	First honest accuracy numbers	data licensing/ acquisition; co-registration effort
4	SIH demo hardening	Offline-key-proofed demo script; warm caches; rehearsal runbook; the "internet-cut" moment	Phase 3 (data), any	A deterministic 5-min demo	live stumbles; demo-only (synthetic) data exposure
5 (future)	Expansion	Real tool chaining (reproject/crop/NDVI) in the graph; local VLM; tiling; multi-temporal; batching; auth tiers	Phase 4 gap review	Multi-tool agent, deeper domain coverage	scope creep; GPU costs

Phases 0–2 are all present in this repository; phases 3–5 are proposed.

18. TEAM IMPLEMENTATION BREAKDOWN (parallel streams)

Stream	Scope	Key artifacts (existing)	Depends on
Frontend	QueryComposer, ImageryViewer (placement/ zoom), Compare, Datasets, Reports, hero scene	<code>frontend/src/app/(app)/*</code> , <code>components/app/*</code> , <code>components/scene/*</code> , <code>hooks/useAnalysisSession.ts</code>	API contract (<code>types.ts</code>)
Backend API	Assets, sessions, workflows, reports, auth, security middleware	<code>app/routers/*</code> , <code>app/middleware/security.py</code>	—
AI/ML	VLM client + prompts + offline fallbacks + model injection points	<code>app/ai/*</code> , <code>app/core/model_provider.py</code>	—
Geospatial	Raster metadata, previews, coordinate transforms	<code>app/geospatial/*</code>	—
Agent orchestration	LangGraph graph, planner, routing, evidence, report, tools	<code>app/agents/*</code> , <code>app/agents/state.py</code> , <code>app/agents/tools.py</code>	AI/ML + Geospatial
Data / persistence	Models, migrations, seed, storage backend	<code>app/models/*</code> , <code>alembic/</code> , <code>scripts/seed_demo.py</code> , <code>app/core/storage.py</code>	Backend API
Integration / testing	Async httpx integration tests, transaction-rollback fixtures, storage patching	<code>tests/*</code> , <code>pytest.ini</code>	all streams

Stream	Scope	Key artifacts (existing)	Depends on
Pitch / demo	Script, runbook, hero narrative (Cartosat-3), seed data press-kit	<code>frontend</code> hero, <code>scripts/generate_sample_geotiffs.py</code>	Integration + Data

Dependency rule of thumb: Geospatial + AI/ML are leaves; Orchestration depends on both; Backend API + Frontend consume Orchestration outputs; Data/persistence serves Backend API; Testing and Pitch run in parallel on stable interfaces. The graph (`orchestrator.py`) is the integration point to freeze first.

19. EVALUATION & SUCCESS METRICS

There are **no accuracy/evaluation numbers in the codebase today**. Everything below is the proposed evaluation contract.

#	Metric	What to measure	How to measure	Datasets/ benchmarks (proposed)
19.1	Intent classification	Accuracy of <code>PlannerSchema.workflow</code> vs. hand-labeled intent	Prompt a held-out set of 40–100 queries; compare label→workflow	<code>QueryComposer</code> suggestions + labeled ISRO-style panels
19.2	Model/tool routing	Correctness of <code>route_after_plan</code> + <code>requires_validation</code> decision	Same as 19.1 + parse-failure rate	recorded query logs
19.3	VQA	Answer correctness + supporting-region validity (relation to scene)	5-point human rubric; region box overlap with human-marked areas	small labeled ISRO-style panel (10–20 scenes)
19.4	Grounding	Detection IoU vs. human boxes per label	Compute IoU in normalized space on <code>box_2d</code> conversion	labeled "building/ water/vegetation" masks
19.5	Change detection	Region hit rate; change-class accuracy; false-alarm rate on unchanged scenes	Compare <code>change_regions</code> vs. known change maps; verify "no change" scenes produce <code>low</code>	synthetic 2022/2026 pair (known footprint) + ±unchanged samples
19.6	SAR fusion	Cross-modal agreement + class accuracy	Per-finding check that <code>optical_evidence</code> and <code>sar_evidence</code> are consistent with <code>fusion_interpretation</code> ; class vs. ground truth	synthetic VV/VH sample + optional real SAR (future)

#	Metric	What to measure	How to measure	Datasets/ benchmarks (proposed)
19.7	Evidence correctness (automated checks)	Completeness: every finding has <code>evidence_refs</code> , <code>confidence</code> , <code>uncertainty_flagged</code> ; report reflects the flag	Add a CI test asserting schema invariants over recorded runs	any run output
19.8	E2E latency	p50/p95 of <code>run_pipeline</code> and per-node duration	<code>track_performance</code> (<code>app/core/metrics.py</code>) aggregated over N runs (currently log-only)	demo dataset
19.9	System reliability	Worker success rate; no-crash rate under missing metadata/keys	Failure-injection + soak runs	chaos of remove-key / bad-CRS / missing-asset cases

20. FINAL PROJECT BLUEPRINT

20.1 Master summary

- **Problem:** satellite imagery analysis is trapped behind specialist tooling and disconnected point-models; answers lack coordinates, evidence, and honesty about uncertainty.
- **Idea:** make image analysis an *agent-first question*. A planner picks the right specialist, the specialist answers in pixels, geospatial code converts to WGS84, and evidence + report make it trustworthy.
- **Solution:** FastAPI + LangGraph + Gemini VLM (with deterministic offline fallback)
- geospatial transform + PostGIS + Next.js — five specialist workflows, one coordinate truth, one evidence chain.
- **Core workflow:** query → plan → specialist → (validate) → evidence → report → georeferenced answer + map overlay + export.
- **Core technologies:** FastAPI, LangGraph, gemini VLM / gpt-4o planner (swappable), rasterio/pyproj/geopandas, PostgreSQL+PostGIS, Redis, MinIO/S3 (local | s3), Docker Compose, Next.js/React/Three.js.
- **Key innovation:** VLM-detected pixels become real WGS84 map features with never-hidden uncertainty and zero-key determinism.
- **MVP:** end-to-end question→georeferenced finding→evidence→report, no API key required, in Docker Compose.
- **Key demo:** "Ask, watch, verify" — grounding + change detection + SAR fusion, with the internet-cut resilience moment.
- **Feasibility:** architectural feasibility HIGH (built); model-quality feasibility MEDIUM (no eval yet; real data injection next).

- **Viability:** real user need, direct friction removal, horizontal-scaling design, mapping to ISRO domains; credibility pending real-data results.
- **Impact:** accessible analysis, faster disaster/agri/urban/infrastructure/environment workflows, auditable outputs.
- **Future scope:** real tool chaining, local VLM, tiling, multi-temporal trends, batch ingestion, multi-user, evaluation harness.

20.2 One-slide architecture

SatQuery AI — Ask. Watch. Verify.

- **Planner (LangGraph):** intent → 1 of 5 specialists + specialist prompt.
- **Specialists:** VQA · Captioning · Grounding · Change Detection · Optical+SAR — all via Gemini 2.5 Flash, all with structured JSON + **deterministic offline fallback**.
- **Geospatial truth:** rasterio (CRS/affine) → pyproj → **WGS84 GeoJSON** with pixel/native/WGS84 boxes.
- **Trust layer:** confidence gate (0.4) → evidence chain → report; warnings never hidden.
- **Stack:** FastAPI + Redis worker + PostGIS + Next.js + nginx + Docker Compose (8 services).

20.3 One-slide solution

One question — a plotted, evidenced answer:

1. Type "Where are the water bodies?" in plain language.
2. The orchestrator plans, routes to Grounding, and shows its work in a live trace.
3. The VLM answers in a 0–1000 pixel space; the geospatial layer converts each box to WGS84 polygons over the image.
4. Confidence, source asset, workflow, and a human-readable report attach to every finding — exportable as JSON/GeoJSON/PDF.
5. Compare mode aligns 2022 vs 2026 with one shared viewer; SAR fusion cites both sensors before it claims.

20.4 One-slide impact

From "read the image" to "answer the mission."

- Disaster response: flood questions answered in minutes; SAR confirms where optical can't.
- Agriculture: vegetation change and NDVI across epochs.
- Urban & infrastructure: build-out measurement; SAR double-bounce for construction evidence.

- Environment & governance: deforestation visibility + auditable, exportable evidence chains.

20.5 One-slide feasibility

Built to demo. Designed to scale. Ready for real data.

- Working today, zero API keys: planner defaults, VLM heuristics, raster pre-processing, PostGIS persistence, frontend overlays, Docker Compose bring-up.
- Gated upgrades: GPT-4o planner and Gemini 2.5 Flash VLM switch on with one key.
- Honest path forward: inject real ISRO/Bhuvan rasters; stand up a labeled evaluation panel; then measure VQA/grounding/change/SAR properly (metrics infra exists).
- No over-claiming: everything in this document is codegrounded; no invented accuracy, partner, or deployment.

21. CONSISTENCY AUDIT

This section is the required self-check: contradictions, unsupported claims, missing implementation details, over-engineered components, MVP-cut candidates, and dependency verification notes.

21.1 Contradictions / inconsistencies in the codebase

- **DUAL BACKEND TREES.** There are two parallel implementations:
 - `app/` — the *active* root-package architecture (FastAPI routers, LangGraph orchestrator, async PostGIS models, worker, Alembic).
 - `satquery_backend/app/` — a **legacy/predecessor** tree (its own FastAPI app, synchronous agents, its own `GeminiVisionClient`, its own Pydantic schemas, and a leftover `satquery_backend/.env`). It is present in the repo, its storage DB (`satquery_backend/storage/satquery.db`) exists, and it is NOT the code docker compose builds. **Recommendation:** delete or archive `satquery_backend/` to prevent drift confusion — this is a housekeeping contradiction, not a semantic one: the two orchestrators use *different* VLM entry points (the legacy one plans via VLM; the active one plans via LLM structured output).
- **Prompt-schema mismatch in fallback vs. schema.** `ORCHESTRATOR_INTENT_PROMPT` (in `app/ai/prompts.py`) still describes the *old* "multimodal_sar_optical" workflow, but the active `PlannerSchema` uses `sar_fusion`. The constant is unused by the active graph (the planner builds its own prompt), but keeping it is misleading. **Recommendation: delete or reconcile.**
- **Evidence model name is a placeholder.** `evidence_agent.py` hardcodes `"provenance": {"model_used": "VLM_PLACEHOLDER"}` — the "actual model name after

injection" documented in its docstring is not implemented. This is fine as a forward contract but must be flagged when stating "real model in provenance."

- **Validation routing vs. the stated "check before evidence".** The graph runs validation **only when** `requires_validation` **is true** (specialist→validation else specialist→evidence) — which is the documented design. But note the dual gate: the *frontend and evidence agent* also treat `confidence < 0.4` as below-threshold. If a specialist returns a mid-run low-confidence finding while the planner did *not* flag validation, the evidence agent still records a `warning`, but no `validation` trace step occurs and `uncertainty_flag` may be false. **Recommendation:** decide whether 0.4 is *always* a gate or only a gate when the planner suspects trouble; make the trace unambiguous either way.
- `handle_low_confidence` / `handle_unsupported_modality` in `workflows.py` are defined as graceful failure handlers with explicit thresholds, but **nothing in the active graph currently calls them** — they belong to the earlier design's orchestrator. Dead-ish helpers.
- `AnalysisRun.duration_ms` **not set by the worker.** The worker passes `duration_ms=None`, and `track_performance` logs a duration but never writes it to the run. The Findings UI therefore shows no duration even though the column exists (`FindingsPanel` reads `result.duration_ms != null`). Minor wiring gap.

21.2 Unsupported claims to avoid

- **Any accuracy number.** None exists. Offline fallbacks return hardcoded confidences (0.82–0.96) — **these are heuristic confidences, not measured accuracy.** The seed report's "12% increase in built-up area" is seed **content** (from `seed_demo.py`), not a measured result.
- **"Cartosat-3"** appears only in the frontend hero scene comments/specs (a *narrative element*). It is not a data source or an integration. Say "modelled on Cartosat-3" or "inspired by."
- **"Sentinel-1 GRD"** appears only in `sar_agent.py`'s *documentation comment* of an accepted input format. The bundled SAR is synthetic. Do not present real Sentinel ingestion as demonstrated.
- **"NDVI"** is a declared tool stub (`compute_ndvi`), not yet routed or demonstrated.
- **"Multi-user workspaces," "JWT hardening," "R2/GCS"** — auth exists functionally but is not battle-tested; S3 backend is proven against local MinIO only.
- **Live agent trace being real-time.** The WS is lossy pub/sub with no replay; the UI correctly admits "Polling — trace socket unavailable." Do not claim guaranteed streaming.
- **docker-compose seed note:** the demo seeder seeds only the optical-2026 + SAR-2026 assets (2), **not** the 2022 optical file. The 2022 file exists in `sample_data/` and `data/raw/`, but is not inserted by `seed_demo.py` — for the change-detection demo, either upload 2022 manually or extend the seeder (a proposed fix).

21.3 Missing implementation details (present-but-thin)

- **load_vlm (HuggingFace)** — fully scaffolded loader with a warning stub path; never invoked by the active pipeline. The docstring is aspirational ("actual model checkpoints are NOT loaded yet").
- **app/agents/tools.py dispatch stubs** — `run_vqa` / `run_grounding` /etc. return `{"status": "dispatched"}` and are not wired into the graph. No LangChain tool invocation is executed today.
- **ModelRun table** — schema exists, never written by the pipeline.
- **Caching** — `app/core/cache.py` helpers + Redis are available; the agent pipeline does not yet cache identical queries/previews in Redis (previews are on-disk cached).
- **duration_ms** — not populated (see 21.1).
- **Auth** — full JWT + refresh + rate limiting implemented in `app/core/auth.py` / `app/routers/auth.py`, but sessions & queries do not enforce login (`auth_required` defaults false). Auth is "available", not "enforced."

21.4 Over-engineered components (for this stage)

- **satquery_backend/ legacy tree** — duplicated functionality; candidate for removal/archive.
- **Full MinIO/S3 + presigned-URL path** — correct design, overkill for the SIH demo where local storage suffices; keep as the documented production path.
- **ModelRun + tools_used** — schema plumbing beyond current need until tool chaining lands.
- **Nginx TLS + Docker secrets ceremony** — already operational; don't expand (self-signed certs acceptable for demo).
- **The WebGL hero scene** — high polish, high effort; ship as-is (it's the visual anchor) but do not grow it.

21.5 Features to remove from the MVP (or de-emphasize)

- **satquery_backend/ tree** — archive.
- Unused prompt constant `ORCHESTRATOR_INTENT_PROMPT` (or reconcile it).
- `handle_low_confidence` / `handle_unsupported_modality` — either wire them in or remove.
- **app/agents/tools.py stubs** — keep only as a documented future extension point; remove the "dispatched" claim semantics.
- Multi-step agent "chaining" story in pitch materials — today it is single-workflow dispatch; say that plainly.
- Any claims that the pipeline is fully reactive on websockets — it is poll+subscribe.

21.6 Dependency verification notes

- **gemini-2.5-flash** — configured default; verify the exact model id is still accepted by `google-genai` on the Google API before demo.
- **torch / transformers** — installed in `requirements.txt` but the local VLM is not used; this is heavy (multi-GB) per-Docker-image cost with no runtime benefit yet. **Proposed:** pin/optionalize for the API/worker images until `load_vlm` is real.
- **gpt-4o (default planner)** — requires `OPENAI_API_KEY`; the stub LLM path works but the planner then *defaults to vqa* (never exercises other workflows via the real LLM). For a multi-workflow demo with an LLM, either supply a key, or ensure switching `LLM_PROVIDER` to `gemini / ollama` is smoke-tested.
- **PostGIS geometry persistence** — `store_workflow_result` writes the first box as a WKT POLYGON; **asset uploads store bbox in the file's own CRS units, not WGS84**, and `assets.list_assets` reads `bbox` via `ST_XMin` etc. — units assumption matters (this matches typical EPSG:32643 composes for the samples). For arbitrary CRS inputs, confirm coordinate-unit handling (`FindingsPanel` explicitly refuses to infer area/units — a correct, documented decision).
- **geoalchemy2 + func.ST_AsGeoJSON** are used; on a fresh database ensure the PostGIS extension exists (the `postgis/postgis:16-3.4` image provides it; the migration must not silently skip geometry columns).
- **redis setex with jsonable_encoder** — fine for small values; do not cache large rasters through it.

21.7 Grounding of key claims (final check)

Claim	Grounding
"Five specialist agents exist"	<code>app/agents/specialists/</code> (vqa, grounding, change, sar, evidence, report)
"LangGraph conditional graph"	<code>orchestrator.py build_graph()</code> ; edges START/plan/.../END
"Confidence threshold 0.4"	<code>CONFIDENCE_THRESHOLD = 0.4</code> (orchestrator); echoed in <code>workflows.py</code> + Findings UI copy
"Pixel→WGS84 GeoJSON"	<code>coordinate_transform.py</code> (affine + pyproj, <code>transform_box_to_geojson</code>)
"Redis async queue + WS pub/sub"	<code>query_worker.py</code> (<code>brpop / publish</code>), <code>sessions.session_websocket</code>
"PostGIS models"	<code>app/models/*</code> (Geometry 4326 fields), PostGIS image in compose
"Offline fallback"	<code>gemini_client._offline_fallback_response</code> + <code>_offline_sar_fusion_fallback</code>
"8 services in compose"	<code>docker-compose.yml</code> (nginx, migrate, api, worker, frontend, db, redis, minio + minio-init)
"S3 assets downloaded to temp"	<code>query_worker.process_query</code> (mkdtemp + <code>download_file</code> + rmtree)
"Preview percentile stretch"	<code>preview_generator.normalize_band_percentile</code> (2–98%)

End of document. Anything not grounded above is marked PROPOSED, FUTURE, or ASSUMPTION and must not be stated as delivered.